The deal.II Library, Version 9.8

Daniel Arndt[1*], Wolfgang Bangerth[2,3], Bruno Blais[4], Marc Fehling[5], Rene Gassmöller[6], Timo Heister[7], Luca Heltai[8], Vladimir Ivannikov[9], Martin Kronbichler[10], Matthias Maier[11], Peter Munch[12], Andreas Steger[13], Dominik Still[14], Bruno Turcksin[1*], and David Wells[15]

[1]Computational Coupled Physics Group, Computational Sciences and Engineering Division, Oak Ridge National Laboratory, 1 Bethel Valley Rd., TN 37831, USA.
arndtd/turcksinbr@ornl.gov
[2]Department of Mathematics, Colorado State University, Fort Collins, CO 80523, USA.
bangerth@colostate.edu
[3]Department of Geosciences, Colorado State University, Fort Collins, CO 80523, USA.
[4]Chemical Engineering High-performance Analysis, Optimization and Simulation (CHAOS) laboratory, Department of Chemical Engineering, Polytechnique Montréal, PO Box 6079, Stn Centre-Ville, Montréal, Québec, Canada, H3C 3A7.
bruno.blais@polymtl.ca
[5]Department of Mathematical Analysis, Faculty of Mathematics and Physics, Charles University, Sokolovská 49/83, 186 75 Prague 8, Czech Republic.
marc.fehling@matfyz.cuni.cz
[6]GEOMAR Helmholtz Centre for Ocean Research Kiel, 24148 Kiel, Germany.
rgassmoeller@geomar.de
[7]School of Mathematical and Statistical Sciences, Clemson University, Clemson, SC, 29634, USA. heister@clemson.edu
[8]Department of Mathematics, University of Pisa, Via Buonarroti 1/c, 56127 Pisa, Italy.
luca.heltai@unipi.it
[9]Institute of Material Systems Modeling, Helmholtz-Zentrum Hereon, Max-Planck-Straße 1, 21502 Geesthacht, Germany. vladimir.ivannikov@hereon.de
[10]Faculty of Mathematics, Ruhr University Bochum, Universitätsstr. 150, 44780 Bochum, Germany. martin.kronbichler@rub.de
[11]Department of Mathematics, Texas A&M University, 3368 TAMU, College Station, TX 77845, USA. maier@math.tamu.edu
[12]Institute of Mathematics, Technical University of Berlin, Germany.
muench@math.tu-berlin.de
[13]TUM School of Engineering and Design, Technical University of Munich, Germany.
andreas.steger@tum.de
[14]Institute for Computational Mechanics, TUM School of Engineering and Design, Technical University of Munich, Germany. dominik.still@tum.de
[15]Department of Mathematics, University of North Carolina, Chapel Hill, NC 27516, USA.
drwells@email.unc.edu

Abstract: This paper provides an overview of the new features of the finite element library deal.II, version 9.8.

1 Overview

deal.II version 9.8.0 was released August 8, 2026. This paper provides an overview of the new features of this release and serves as a citable reference for the deal.II software library version 9.8. deal.II is an object-oriented finite element library used around the world in the development of finite element solvers. It is available under the terms of the *Apache License 2.0*[*] *with LLVM Exception*[†] and separately under the terms of the *GNU Lesser General Public License v2.1*.[‡] Downloads are available at https://www.dealii.org/ and https://github.com/dealii/dealii.

The major changes of this release are:

- Substantially extended support for meshes that contain triangles, tetrahedra, wedges, and pyramids (see Section 2.1).
- Improved support for and performance on GPUs (see Section 2.2).
- Support for newer versions of Trilinos that build on its newer Tpetra, rather than the older Epetra, foundation (see Section 2.3).
- Much extended interfaces to the VTK library (see Section 2.4) and the SUNDIALS library (see Section 2.5).
- A lot of work on reducing memory allocations to improve performance (see Section 2.6).
- Continued evolution alongside newer C++ standards (see Section 2.7).
- A substantial number of new tutorial and code gallery programs (see Section 2.8).

While all of these major changes are discussed in detail in Section 2, there are a number of other noteworthy changes in the current deal.II release, which we briefly outline in the remainder of this section:

- Major parts of the `MeshWorker` framework are now deprecated. When originally written, more than 20 years ago, `MeshWorker` was an attempt at automatizing many parts of what a finite element code does. It succeeds at making easy things easier, but makes it very difficult to transition, for example, from a discretization of a simple PDE without coefficients to a real problem with nonlinearities, spatially variable coefficients, and similar things. It was also never well documented or tested, and only `step-39` heavily relied on it. `step-39` has now been rewritten, and all of the classes it used are now deprecated. The only piece within the `MeshWorker` namespace that will survive in the long run is the `MeshWorker::mesh_loop()` function and its dependencies.
- Our wrappers of PETSc's linear algebra classes now support multi-threaded assembly into and reading from vectors ghost.
- We have extended the matrix-free infrastructure for the evaluation and integration of Nédélec elements.
- The internal representation of imposing Dirichlet boundary conditions in the matrix-free infrastructure was reworked and made more similar between the GPU and CPU implementations, leading to faster execution on most CPU systems.

The changelog – listing more than 140 new features and bugfixes – contains a complete record of all changes; see [46].

[*] https://spdx.org/licenses/Apache-2.0.html
[†] https://spdx.org/licenses/LLVM-exception.html
[‡] https://spdx.org/licenses/LGPL-2.1-or-later.html

2 Major changes to the library

This release of deal.II contains a number of large and significant changes, which we will discuss in the following.

2.1 Support for meshes with non-hypercube cells

We have continued deal.II's transition from a library that historically only supported hypercube cells to one that also supports triangular (in 2d), tetrahedral, pyramidal, and wedge-shaped cells (in 3d) meshes, as well as mixed meshes. In particular, we have put substantial effort into supporting more finite element types on these meshes.

The pyramid finite elements FE_PyramidP and FE_PyramidDGP now support quadratic and cubic shape functions, extending the mixed-mesh capabilities introduced in release 9.3 [8] to higher polynomial orders. The underlying approximation space is constructed by transforming the orthogonal basis of Bergot et al. [16] into a nodal Lagrange basis. Analogous constructions are implemented for simplices and wedges. The orthogonal basis functions are expressed in terms of (homogenized) Jacobi polynomials and evaluated using recurrence relations rather than explicit polynomial expressions. Derivatives are evaluated using the same recurrence-based approach, avoiding computations at singularities while improving numerical stability. The pyramid basis functions form an exception, as their approximation space is spanned by rational rather than polynomial functions.

The introduction of second- and third-order nodal pyramid elements requires support for degrees of freedom located on edges and faces for mixed element types. To ensure a consistent distribution of the DoFs across interfaces, the *hp*-infrastructure has been extended to account for the different face topologies on transitional elements, thereby preserving continuity on conforming meshes.

Increasing the polynomial degree of the shape functions also requires quadrature rules of sufficiently high order to consistently evaluate weak-form integrals, particularly in the presence of non-linear terms or curvilinear mappings. To this end, mixed-mesh computations are supported by Stroud quadrature rules. These rules provide integration formulas of arbitrary order and serve as a fallback whenever no alternative quadrature rule of sufficient accuracy is available.

Additionally, isotropic refinement is now supported for pyramids and wedges. The underlying algorithms employ a midpoint refinement strategy similar to the one used for hexahedra and tetrahedra. It recursively subdivides a cell's constituent lower-dimensional entities – from lines to faces, and finally to the element itself. This hierarchical approach ensures mesh conformity across neighboring elements. Under this scheme, an isotropic refinement step partitions a pyramid into ten and a wedge into eight children.

As an alternative to working directly with simplex meshes, deal.II now provides the function GridGenerator::convert_simplex_to_hypercube_mesh(), which converts a simplex mesh into a pure hypercube mesh. In two dimensions, each triangle is subdivided into three quadrilaterals by inserting a vertex at the barycenter and connecting it to the edge midpoints. In three dimensions, each tetrahedron is decomposed into four hexahedra by applying the same subdivision to every face and adding a vertex at the tetrahedron's barycenter.

To enable the evaluation of finite element fields across interfaces in mixed meshes, the class FEInterfaceValues now supports heterogeneous element types. The two sides of a face or subface may use different finite elements, mappings, and quadrature rules, even allowing for adaptively refined mixed meshes in two dimensions.

Table 1: *Stokes operator throughput and time to solve the system for different devices. PMF denotes the new* `Portable::MatrixFree` *implementation optimized for GPUs, and MF denotes the CPU-based* `MatrixFree` *framework leveraging SIMD. MF_C exploits the fact that meshes are Cartesian. Results from step-104, on a problem with 53 million degrees of freedom, in 3D.*

Device	**Type**		**FP64** [TFLOPS]	**VRAM** [GB]	**BW** [TB/s]	**Operator** [MDoFs/s]	**Solve Time** [s]
AMD W7800	GPU	PMF	1.41	32	0.6	469	31
NVIDIA RTX 6000	GPU	PMF	1.97	96	1.8	1,504	10
NVIDIA H100	GPU	PMF	25.6	80	3.4	1,457	9
AMD 5975WX	CPU	PMF	2.0	–	0.2	113	135
AMD 5975WX	CPU	MF	2.0	–	0.2	995	21
AMD 5975WX	CPU	MF_C	2.0	–	0.2	1,873	13
2x AMD EPYC 9754	CPU	PMF	12.7	–	0.92	654	26
2x AMD EPYC 9754	CPU	MF	12.7	–	0.92	4,885	5
2x AMD EPYC 9754	CPU	MF_C	12.7	–	0.92	12,129	3

2.2 Performance Portable GPU support

This release comes with numerous new features, fixes, and performance improvements for running on GPUs, making GPUs usable in practice to speed up computations.

We build on Kokkos [64] for performance portability, meaning a code written based on the `Portable::MatrixFree` classes can run on current GPUs from NVIDIA, AMD, and Intel without any vendor-specific code visible in the user code. Importantly, all functionality is also available on the CPU if `deal.II` is configured without GPU support.

A representative use case is the new `step-104` tutorial program (see also Section 2.8), that solves the Stokes equation using a block preconditioner and an inner geometric multigrid V-cycle. The operator evaluation and multigrid cycle is running without assembling any matrices (i.e., "matrix-free"). Performance results for a variety of GPUs and CPUs are presented in Table 1. Here, "PMF" denotes the new portable matrix-free framework optimized for GPUs. The performance on CPUs is significantly lower than the original matrix-free framework optimized for CPUs (denoted by "MF"). This difference is partly explained by the fact that the PMF implementation is not using SIMD vectorization, and the MF implementation has been optimized specifically for usage on CPUs over the years. The portable framework can currently also not take advantage of structured grids (like assuming the determinant of the Jacobian of the mapping being constant within a cell; "MF_C" denotes that these optimizations being used). A comparison with server-grade AMD CPUs with the H100 setup, which formally have a lower peak FP64 rate and lower memory bandwidth, indicate further potential for the GPU implementation. Future work will therefore consider further performance optimization for the portable framework and taking advantage of Cartesian meshes.

Similar to the `MatrixFree` framework designed for CPUs [43, 44], throughput for operator evaluation increases with polynomial degree. This is in contrast to matrix-vector products using assembled matrices that get slower the higher the finite element degree, as more entries per row are nonzero. As GPUs are highly parallel devices, a significant number of unknowns per GPU are required to reach peak performance, as is clear from Figure 1.

The new tutorial showcases various new features of `deal.II`: First, `Portable::MatrixFree` now supports systems of equations by allowing for the use of more than one `Portable::FEEvaluation` object within an operator. In `step-104`, this is used to define the Stokes operator with a velocity and pressure evaluation object using the same quadrature rule. Second, we overhauled the

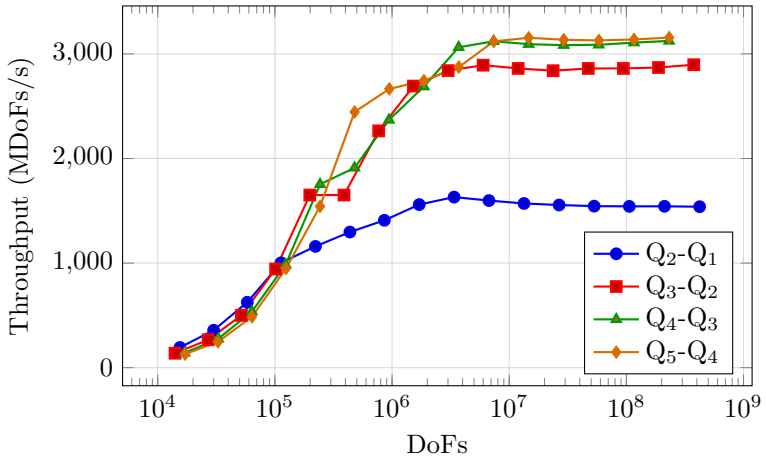

Figure 1: *Throughput of a Stokes operator application for different problem sizes and finite element degrees on an NVIDIA RTX 6000 Blackwell Max-Q GPU. The* `step-104` *program provides a detailed description and further performance data.*

interface by providing `DeviceVector` and `DeviceBlockVector` classes within the matrix-free GPU kernels instead of exposing raw C-style pointers. Third, the complete geometric multigrid cycle including multigrid transfer for *h* and *p* coarsening runs on device without copying solution data between host and GPU, even when running with multiple GPUs. This is achieved using device-aware MPI support. Fourth, performance has been significantly improved through code optimizations, reducing the number of GPU fences, kernel fusion where possible, and by using multithreading for setup steps that run on the CPU.

Finally, usability improved with various additional error checks throughout the interface, improved API documentation, and other new features. Taken together, the work over the last twelve months makes the portable matrix-free framework a competitive solution for performance critical `deal.II` applications.

2.3 Trilinos and Tpetra interfaces

The Trilinos project is an important dependency for `deal.II`, providing one way to parallel linear algebra classes as well as numerous other pieces of functionality like solvers and preconditioners. Historically, Trilinos has provided parallel linear algebra functionality through its `Epetra` subpackages. However, the Trilinos project has long intended to replace their `Epetra`-based solver stack in favor of `Tpetra`-based solvers. Support for different number types and `Kokkos` execution spaces have been the main drivers of this transition, the latter also enabling support for GPUs or other acceleration devices. The `Epetra`-based stack was finally removed in the recent Trilinos 17 release. Because `deal.II`'s Trilinos wrappers were all based on `Epetra`, this has required us to complete the `Tpetra` wrappers we had started to write in recent years; for example, the `TpetraWrappers::Vector` class has been available since `deal.II` 9.1, and a basic implementation for `TpetraWrappers::SparsityPattern` and `TpetraWrappers::SparseMatrix` was included in the 9.6 `deal.II` release. However, much functionality was still missing, and in particular `deal.II` would not configure and compile with Trilinos if the `Epetra` stack was not present.

With `deal.II` 9.8 `Tpetra` support was expanded significantly, while simultaneously improving the compatibility between `Tpetra` and `Epetra` based wrapper classes. We now support Trilinos versions that include the `Epetra`-based stack, the `Tpetra`-based stack, or both; in particular, this includes Trilinos' latest release 17 that no longer has `Epetra` support. In case `Epetra` is available, the `TrilinosWrappers` namespace continues to use `Epetra` but will use `Tpetra` if only `Tpetra`

is available. The TpetraWrappers namespace will always use Tpetra and is only available if Trilinos was configured with Tpetra.

While we generally expect the Tpetra stack to work, not all functionality from namespace TrilinosWrappers is available in the TpetraWrappers namespace at this time. However, in practice we built very large packages with the Tpetra wrappers, indicating that all *major* functionality is available. On the other hand, a substantial number of our Trilinos wrapper tests and several of deal.II's example programs fail when run against the Tpetra wrappers, suggesting that some functionality available in our Epetra wrappers are not yet implemented in the Tpetra wrappers.

2.4 VTK interfaces

The interfaces in the VTKWrappers namespace have been substantially extended. The new functionality provides a bidirectional bridge between VTK data sets and the mesh, finite-element, and vector abstractions used by deal.II. In particular, VTK files can now be read in stages: read_tria() constructs a Triangulation, while read_cell_data() and read_vertex_data() expose data associated with cells and vertices, respectively. The corresponding read_all_data() utility collects all data arrays into a single serial vector, and read_vtk() provides a combined interface for loading a VTK file and its associated finite-element data.

The description of the data in a VTK file can also be translated into a compatible deal.II finite-element object with vtk_to_finite_element(). This makes it possible to preserve the number and location of the degrees of freedom represented in the file without imposing an interpretation on multi-component arrays. Once a DoFHandler has been constructed, data_to_dealii_vector() maps the VTK data onto a deal.II vector, including distributed vectors on a parallel triangulation. The supporting GridTools::parallel_to_serial_vertex_indices() utility supplies the vertex-numbering correspondence required when data are read on a serial mesh and then transferred to a distributed one.

Finally, the VTK interface now supports two-way conversion between a Triangulation and a vtkUnstructuredGrid. This allows meshes to be exchanged with VTK-based visualization and meshing tools in both directions, including meshes with non-hypercube reference-cell types supported by the library. The conversion optionally transfers manifold, boundary, and material identifiers associated with the deal.II mesh. Together, these additions make it possible to use VTK files as input for restarting or preprocessing computations while retaining the topology and metadata needed by a finite-element program.

2.5 Interfaces to the SUNDIALS ARKODE package

The interfaces to the SUNDIALS ARKODE package have been substantially reorganized, now matching the internal organization of ARKODE. The SUNDIALS::ARKode class now acts as a common driver for time integration, while stepper-specific setup and callbacks have been moved into a new ARKodeStepper hierarchy. This mirrors the modular structure of ARKODE itself and removes the previous assumption that all time integration is performed through ARKODE's ARKStep module. The existing implicit-explicit functionality is now provided by ARKStepper. To smooth the migration to the new class hierarchy, deprecated compatibility wrappers have been introduced that preserve the old interface.

This restructuring made it possible to add wrappers for further ARKODE modules. The new ERKStepper exposes the explicit ERKStep module for non-stiff problems. The LSRKStepperSTS and LSRKStepperSSP classes expose the low-storage Runge–Kutta LSRKStep module for stabilized explicit integration of mildly stiff parabolic problems, and strong-stability-preserving integration of hyperbolic or advection-dominated problems, respectively.

The LSRKStepperSTS stepper requires an estimator for the dominant (largest-magnitude) eigenvalue of the Jacobian of the ODE right-hand side to determine the number of polynomial stages to

Table 2: *Total number of allocations performed (with one MPI process and one thread) by several tutorial programs in 9.7.1 and 9.8.0. For* `step-32` *the program was run to* 1/100*th of its default final time; all other time-dependent problems were run to* 1/10*th of their default final times.*

Tutorial	Description	Allocations		
		deal.II 9.7.1	deal.II 9.8.0	
`step-23`	adaptive wave equation solver	2,871k	623k	−78%
`step-32`	adaptive time-dependent Stokes solver	26,847k	11,791k	−56%
`step-35`	projection solver for Navier-Stokes	4,408k	975k	−78%
`step-40`	adaptive Laplace solver	5,265k	2,028k	−61%

be used. By default, the power iteration method implemented in the `SUNDomEigEstimator_Power` module of `ARKODE` is employed to compute an estimate of the dominant eigenvalue. Users can replace this estimator with one of their choice.

2.6 Reducing dynamic memory allocations

We made a significant effort to reduce both the memory usage and number of memory allocations done by `deal.II`. We used heaptrack [66] to detect and measure the number of dynamic allocations. Significant improvements include:

- lowered memory consumption of purely tetrahedral `Triangulation`s by 20% by using `ReferenceCell`-dependent strides for its internal data structures,
- lowered run time to construct a `DynamicSparsityPattern` for `DoFHandler`s with a single component by 15%,
- movement of all internal allocation functions in `Triangulation` to internal helper classes, which also eliminated some duplicated loops over allocated cells.

We achieved these reductions by preferring flat data structures such as `Table` over `std::map`, moving temporary arrays into higher scopes, using `boost::container::small_vector` instead of `std::vector`, and eliminating temporaries by combining intermediate steps. A short quantitative analysis of various example steps shows a reduction in dynamic allocations between 56% and 78%; see Table 2.

2.7 Work towards more modern C++ standards

As in most previous releases, we have spent substantial work on evolving our code base to keep track with developments in C++ and the standards that define C++.

Specifically, we have continued our work on building C++20-style modules (see the discussion in the previous release paper [7] and the preprint [12]). Recent compiler versions have become stricter at enforcing requirements spelled out in the standard for how the source files for modules have to be structured, and the scripts we use to convert the `deal.II` source files to a module structure have correspondingly evolved. We have also substantially expanded the module wrappers for the external projects we interface with. Note that `deal.II` continues to only require C++17, but can build C++20-style modules if so instructed and if the compiler supports it.

The C++26 standard was released in March 2026 [38]. Among its many additions is a class `std::inplace_vector<T,N>` that has a size that can vary at runtime, but can not exceed `N`, allowing for efficient allocation of such objects either on the stack or inside another class. In contrast, the widely used class `std::vector<T>` has to allocate memory on the heap dynamically,

whereas `std::array<T,N>` has a fixed size `N`. The new class is useful in contexts where the actual size of the object is not known at compile time, but where it is known that the size can not exceed a certain number. For example, during construction of a `Triangulation`, `deal.II` creates a `CellData` object for every cell on the coarsest level. The primary information these objects store is the vertices which define the cell. In 3d, the number of vertices must be between 4 (for tetrahedra) and 8 (for hexahedra), so by storing the vertices in a `std::inplace_vector<unsigned int, 8>` we avoid separate heap allocations for each cell and instead store all data contiguously in memory.

We have written a fallback version of this class which is compatible with C++17 and only omits the small number of features which require newer language support (such as support for ranges and more `constexpr` support). We have started to use it in many places where the maximal size of an array is known at compile time, and this has dramatically reduced the number of dynamic memory allocations in many cases – see also the discussion in Section 2.6.

2.8 New and improved tutorials and code gallery programs

Many of the `deal.II` tutorial programs were revised in a variety of ways as part of this release: Nearly 100 of the more than 1500 (non-merge) commits that went into this release touched the tutorial. In addition, there are a number of new tutorial programs:

- `step-98` shows how to solve two-dimensional magnetostatic curl-curl problems. It complements `step-97`, which considers similar problems in 3d, but `step-98` puts specific emphasis on the interpretation vectors, the curl, and other operations in 2d settings. `step-98` was written by Siarhei Uzunbajakau.
- `step-100` introduces the Discontinuous Petrov–Galerkin (DPG) method, a finite element framework in which the test space differs from the trial space. Using broken (discontinuous) test spaces, optimal test functions are computed element-by-element so that the discrete problem inherits the inf-sup stability of the continuous one (see [23, 24, 25] for a full description of the method). We apply the method to the time-harmonic Helmholtz equation, a problem for which standard Galerkin discretizations lose stability at high wavenumbers, whereas DPG remains robust independently of the wavenumber-mesh regime. Because DPG is equivalent to a residual minimization in a dual norm, the resulting linear system is Hermitian positive definite despite the indefiniteness of the Helmholtz operator, and can therefore be solved with a conjugate gradient solver. Furthermore, the residual provides a built-in a posteriori error estimator that can be used to drive adaptive mesh refinement. This comes at the cost of additional interface unknowns (traces and face elements); the program uses local condensation at the element level to obtain a global system in which only the trace degrees of freedom are present. `step-100` was written by Oreste Marquis, with help and guidance from Bruno Blais and Matthias Maier.
- `step-104` solves the Stokes problem on GPUs using a block preconditioner and a matrix-free geometric multigrid preconditioner; see Section 2.2 for more details. `step-104` was written by Quang Hoang and Timo Heister.

There are also several new programs in the code gallery (a collection of user-contributed programs that often solve more complicated problems than tutorial programs, and that are intended as starting points for further research rather than as teaching tools):

- "*Parallel implementation of heat equation*" is a transient heat equation solver with adaptive mesh refinement and a parallel solver. This program was written by Wasim Niyaz Munshi, Chandrasekhar Annavarapu, and Wolfgang Bangerth.
- "*L-BFGS phasefield solver*" is a limited-memory BFGS monolithic solver for phase-field crack simulations. This program was written by Tao Jin.

– "*Agglomeration-based solver for the Poisson problem*" is a discontinuous Galerkin solver for the Poisson problem on general polytopal meshes generated through mesh agglomeration. This program was written by Marco Feder, Pasquale Claudio Africa, Xinping Gui, and Andrea Cangiani.

– "*An ALE approach for large-deformation thermoplasticity*" is an implementation of a large-deformation arbitrary Lagrangian-Eulerian finite-strain thermoplasticity solver. This program was written by Maien Hamed.

– "*Parallel Vibroacoustic Solver*" is a parallel frequency-domain vibroacoustic solver calculating the sound transmission loss of a concrete wall. This program was written by Andreas Hegendörfer.

– "*Multilevel Monte Carlo for random Darcy flow*" is an implementation of a multilevel Monte Carlo method for random Darcy flow. This program was written by Jonas Plank.

– "*Parallel flow routing*" solves the problem of determining how much water is where in a landscape given by a digital elevation model, using massively parallel algorithms. This program was written by Wolfgang Bangerth.

2.9 Incompatible changes

The 9.8 release includes around 20 incompatible changes; see [46]. Many of these incompatibilities remove previously deprecated functions or classes, or require now widely available but newer versions of external dependencies than previous `deal.II` versions. Others change internal interfaces that are not usually used in external applications. The following are worth mentioning since they are more broadly visible:

– Most of the parallel vector classes (in particular those wrapping the functionality in the PETSc, Trilinos Epetra, and Trilinos Tpetra packages) observe semantics whereby vectors that have ghost elements are treated as unchangeable. This is because whenever you change a locally-owned element of such a vector, this change is not propagated to those processes that store this element among their ghost elements; these processes' view of the vector is now no longer in sync with the view on that process that owns the element, and that is a common source of very hard to find bugs.

  As a consequence, many vector class member functions that modify the vector check that the vector has no ghost elements. But some failed to do so, such as `operator+=()`, `operator-=()`, `operator*=()`, and `operator/=()` functions of the `TrilinosWrappers::MPI::Vector` class, and the corresponding functions in the Epetra and Tpetra wrappers, along with an assortment of other non-`const` functions in these classes that include assigning a scalar to all elements of a vector, and writing into (or adding, or multiplying/dividing) a single vector entry.

  This oversight has now been fixed. We know that some of these functions are used in parallel contexts in application codes that will now no longer work and will need to be fixed. For this, you will want to perform the modifying operations on fully-distributed vectors (i.e., vectors of the same type that store no ghost elements) and then copy the result into the ghosted vector.

– Several of the classes used in the construction of triangulations, specifically `CellData` and `TriangulationDescription::CellData` use the new `std_cxx26::inplace_vector` class (see Section 2.7) for member variables to reduce the number of memory allocations. In practice, because both this and the previously used `std::vector` class use array syntax, few codes will notice the difference in type.

- The `FESubfaceValues` class – used to integrate over the children of a face of a cell – used to do just that: Consider a part of a face that corresponds to a subdivision of that face due to refinement. It did not actually check consistently whether the face *was* indeed refined (i.e., whether the neighboring *cell* was refined). It will now generate an error if one tries to integrate over a face that is not actually refined. In practice, there should not be any reasons why one would integrate over sub-faces that are not in fact refined.
- In three space dimensions, the curl of a vector field is unambiguously a three-dimensional vector. In two space dimensions, it is typically defined as a scalar, interpreted as the magnitude of a vector aligned with the z-axis that results from taking the curl of a vector field that lives in the x-y plane. For historical reasons, `deal.II` used to represent this scalar as a `Tensor<1,1>` (i.e., a tensor of rank one -– a tensor with one index –- for which the only possible value of the index is zero). This is, of course, entirely equivalent to it being a scalar, but it is semantically wrong: The data type should have been a scalar, not a vector of length one. This is now fixed: The data type defined by `FEValuesViews::Vector::curl_type` used for curls is now simply a scalar value for `dim==2`. For `dim==3`, it is unchanged from what it was before.

3 How to cite `deal.II`

In order to justify the work the developers of `deal.II` put into this software, we ask that papers using the library reference one of the `deal.II` papers. This helps us justify the effort we put into this library.

There are various ways to reference `deal.II`. To acknowledge the use of the current version of the library, **please reference the present document**. For up-to-date information and a bibtex entry see

`https://www.dealii.org/publications.html`

The original `deal.II` paper containing an overview of its architecture is [14], and a more recent publication documenting `deal.II`'s design decisions is available as [9]. If you rely on specific features of the library, please consider citing any of the following:

- For geometric multigrid: [41, 39, 20, 51];
- For distributed parallel computing: [13];
- For *hp*-adaptivity: [15, 27];
- For partition-of-unity (PUM) and finite element enrichment methods: [22];
- For matrix-free and fast assembly techniques: [43, 44];
- For computations on lower-dimensional manifolds: [26];
- For curved geometry representations and manifolds: [35];
- For integration with CAD files and tools: [36];
- For boundary element computations: [33];
- For the `LinearOperator` and `PackagedOperation` facilities: [48, 49];
- For uses of the `WorkStream` interface: [65];
- For uses of the `ParameterAcceptor` concept, the `MeshWorker::ScratchData` base class, and the `ParsedConvergenceTable` class: [57];
- For uses of the particle functionality in `deal.II`: [31].
- For the design of the video lectures and how they can be used in teaching: [67].

`deal.II` can interface with many other libraries:

- ADOL-C [34]
- ArborX [54]
- ARPACK [45]
- Assimp [59]
- BLAS, LAPACK [4]
- Boost [18]
- CGAL [61]
- Gmsh [32]
- GSL [29]
- Ginkgo [5, 6]
- HDF5 [62]
- Kokkos [64]
- Magic Enum [47]
- METIS [42]
- MUMPS [3, 2]
- muparser [52]
- OpenCASCADE [53]
- p4est [19]
- PETSc [10, 11]
- PSBLAS [28]
- ROL [40]
- ScaLAPACK [17]
- SLEPc [56]
- SUNDIALS [30]
- SymEngine [60]
- Taskflow [37]
- TBB [55]
- Trilinos [50, 63]
- UMFPACK [21]
- VTK [58]
- zlib [68]

Please consider citing the appropriate references if you use interfaces to these libraries.

The two previous releases of `deal.II` can be cited as [1, 7].

4 Acknowledgments

`deal.II` is a worldwide project with dozens of contributors around the globe. Other than the authors of this paper, the following people contributed code to this release:

> Daniel Abele, Mathias Anselmann, Kyle Arnold, Zdeněk Bonaventura, Julian Brotz, Michele Bucelli, Praveen Chandrashekar, Jerett Cherry, Vishnu Datha, Raksha Devi, Vivienne Ehlert, Marco Feder, Federico Fernandez, Mohamad Ghadban, Quang Hoang, Tyree Huang, Yimin Jin, Jalil Khan, Felix Kimmerle, Sebastian Kinnewig, Katharina Kormann, Damien Lebrun-Grandie, Lukáš Lejdar, Anna Long, Oreste Marquis, Harsha Matta, Ryan Moulday, Natalia Nebulishvili, Manikandan Nishanth, Jonas Plank, Davide Polverino, Ivan Prusak, Ce Qin, Alexander Reinhold, Sam Scheuerman, Torsten Schmid, Magdalena Schreter-Fleischhacker, Florian Schulze, Richard Schussnig, Gordan Segon, Qingyuan Shi, Wyatt Smith, Simon Sticko, Jan Philipp Thiele, Pasquale Tricarico, Siarhei Uzunbajakau, Quentin Viville, Yiliang Wang, Zhaokun Wang, Ivy Weber, Michał Wichrowski, Jiaqi Zhang, Ran Zhang.

That is, 67 people in total contributed to this release. Their contributions are much appreciated – this project lives by its community of users and contributors!

`deal.II` and its developers are financially supported through a variety of funding sources:

D. Arndt and B. Turcksin: Research sponsored by the Laboratory Directed Research and Development Program of Oak Ridge National Laboratory, managed by UT-Battelle, LLC, for the U. S. Department of Energy and supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Next-Generation Scientific Software Technologies program, under contract number DE-AC05-00OR22725.

W. Bangerth was partially supported by the National Science Foundation under awards EAR-1925595 and OAC-2410847.

W. Bangerth, T. Heister, and R. Gassmöller were partially supported by the Computational Infrastructure for Geodynamics initiative (CIG), through the National Science Foundation (NSF) under Award No. EAR-2149126 via The University of California – Davis.

B. Blais was supported by the National Science and Engineering Research Council of Canada (NSERC) through the RGPIN-2020-04510 Discovery Grant and the MMIAOW Canada Research

Chair Level 2 in Computer-Assisted Design and Scale-up of Alternative Energy Vectors for Sustainable Chemical Processes (CRC-2022-00340).

M. Fehling was supported by the ERC-CZ grant LL2105 CONTACT and the P JAC grant CZ.02.01.01/00/22_008/0004591 FerrMion, both funded by the Czech Ministry of Education, Youth and Sports, as well as by the Czech Science Foundation project No. 26-21877S and the Charles University Research Centre Program No. UNCE/24/SCI/005.

R. Gassmöller was partially supported by the Helmholtz Association's Impulse and Networking Fund as part of the IDS_29 ASPECT-NEXT project in Science Serve Call 2025.

T. Heister was also partially supported by NSF Awards OAC-2607668, OAC-2410848, EAR-1925575.

L. Heltai acknowledges the MIUR Excellence Department Project awarded to the Department of Mathematics, University of Pisa, CUP I57G22000700001, and partial support by King Abdullah University of Science and Technology Research Funding (KRF) under Award No. ORFS-2025-CRG13-6911.3. L. Heltai is also member of Gruppo Nazionale per il Calcolo Scientifico (GNCS) of Istituto Nazionale di Alta Matematica (INdAM).

M. Kronbichler was partially supported by the German Federal Ministry of Research, Technology and Space, project "PDExa: Optimized software methods for solving partial differential equations on exascale supercomputers", grant agreement No. 16ME0637K.

M. Kronbichler and L. Heltai were partially supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (call HORIZON-EUROHPC-JU-2023-COE-03, grant agreement No. 101172493 "dealii-X: an Exascale Framework for Digital Twins of the Human Body").

M. Maier was partially supported by NSF Award DMS-2045636 and by the Air Force Office of Scientific Research under grant/contract number FA9550-23-1-0007.

D. Still was partially supported by the German Federal Ministry of Research, Technology and Space, project "PDExa: Optimized software methods for solving partial differential equations on exascale supercomputers", grant agreement No. 16ME0638 and the European Union – NextGenerationEU and by BREATHE, an ERC-2020-ADG Project, Grant Agreement ID 101021526.

D. Wells was supported by NSF Award OAC-1931516.

This research used in part resources on the Palmetto Cluster at Clemson University under National Science Foundation awards MRI 1228312, II NEW 1405767, MRI 1725573, and MRI 2018069. The views expressed in this article do not necessarily represent the views of NSF or the United States government.

This research was supported by grants from NVIDIA and utilized NVIDIA products.

References

[1] P. C. Africa, D. Arndt, W. Bangerth, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, S. Kinnewig, M. Kronbichler, M. Maier, P. Munch, M. Schreter-Fleischhacker, J. P. Thiele, B. Turcksin, D. Wells, and V. Yushutin. The `deal.II` library, version 9.6. *Journal of Numerical Mathematics*, 32(4):369–380, 2024.

[2] P. R. Amestoy, A. Buttari, J.-Y. L'Excellent, and T. Mary. Performance and scalability of the block low-rank multifrontal factorization on multicore architectures. *ACM Transactions on Mathematical Software*, 45(1):2, 2019.

[3] P. R. Amestoy, I. S. Duff, J.-Y. L'Excellent, and J. Koster. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.

[4] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, third edition, 1999.

[5] H. Anzt, T. Cojean, Y.-C. Chen, G. Flegar, F. Göbel, T. Grützmacher, P. Nayak, T. Ribizel, and Y.-H. Tsai. Ginkgo: A high performance numerical linear algebra library. *Journal of Open Source Software*, 5(52):2260, 2020.

[6] H. Anzt, T. Cojean, G. Flegar, F. Göbel, T. Grützmacher, P. Nayak, T. Ribizel, Y. M. Tsai, and E. S. Quintana-Ortí. Ginkgo: A modern linear operator algebra framework for high performance computing. *ACM Transactions on Mathematical Software*, 48(1):2, 2022.

[7] D. Arndt, W. Bangerth, M. Bergbauer, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, M. Kronbichler, M. Maier, P. Munch, S. Scheuerman, B. Turcksin, S. Uzunbajakau, D. Wells, and M. Wichrowski. The `deal.II` library, version 9.7. *Journal of Numerical Mathematics*, 33:403–415, Nov. 2025.

[8] D. Arndt, W. Bangerth, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, U. Köcher, M. Kronbichler, M. Maier, P. Munch, J.-P. Pelteret, S. Proell, K. Simon, B. Turcksin, D. Wells, and J. Zhang. The `deal.II` library, version 9.3. *Journal of Numerical Mathematics*, 29(3):171–186, 2021.

[9] D. Arndt, W. Bangerth, D. Davydov, T. Heister, L. Heltai, M. Kronbichler, M. Maier, J.-P. Pelteret, B. Turcksin, and D. Wells. The deal.II finite element library: Design, features, and insights. *Computers & Mathematics with Applications*, 81:407–422, 2021.

[10] S. Balay, S. Abhyankar, M. F. Adams, S. Benson, J. Brown, P. Brune, K. Buschelman, E. Constantinescu, L. Dalcin, A. Dener, V. Eijkhout, J. Faibussowitsch, W. D. Gropp, V. Hapla, T. Isaac, P. Jolivet, D. Karpeev, D. Kaushik, M. G. Knepley, F. Kong, S. Kruger, D. A. May, L. C. McInnes, R. T. Mills, L. Mitchell, T. Munson, J. E. Roman, K. Rupp, P. Sanan, J. Sarich, B. F. Smith", H. Suh, S. Zampini, H. Zhang, H. Zhang, and J. Zhang. PETSc/TAO users manual. Technical Report ANL-21/39 - Revision 3.23, Argonne National Laboratory, 2025.

[11] S. Balay, S. Abhyankar, M. F. Adams, S. Benson, J. Brown, P. Brune, K. Buschelman, E. M. Constantinescu, L. Dalcin, A. Dener, V. Eijkhout, W. D. Gropp, V. Hapla, T. Isaac, P. Jolivet, D. Karpeev, D. Kaushik, M. G. Knepley, F. Kong, S. Kruger, D. A. May, L. C. McInnes, R. T. Mills, L. Mitchell, T. Munson, J. E. Roman, K. Rupp, P. Sanan, J. Sarich, B. F. Smith, S. Zampini, H. Zhang, H. Zhang, and J. Zhang. PETSc Web page. `https://petsc.org/`, 2025.

[12] W. Bangerth. Experience converting a large mathematical software package written in C++ to C++20 modules. *arXiv preprint*, 2025. `https://arxiv.org/abs/2506.21654`.

[13] W. Bangerth, C. Burstedde, T. Heister, and M. Kronbichler. Algorithms and data structures for massively parallel generic adaptive finite element codes. *ACM Transactions on Mathematical Software*, 38(2):14/1–28, Dec. 2012.

[14] W. Bangerth, R. Hartmann, and G. Kanschat. deal.II — a general purpose object oriented finite element library. *ACM Transactions on Mathematical Software*, 33(4):24, 2007.

[15] W. Bangerth and O. Kayser-Herold. Data structures and requirements for *hp* finite element software. *ACM Transactions on Mathematical Software*, 36(1):4, 2009.

[16] M. Bergot, G. Cohen, and M. Durufle. Higher-order finite elements for hybrid meshes using new nodal pyramidal elements. *Journal of Scientific Computing*, 42:345–381, 2010.

[17] L. S. Blackford, J. Choi, A. Cleary, E. D'Azevedo, J. Demmel, I. Dhillon, J. Dongarra, S. Hammarling, G. Henry, A. Petitet, K. Stanley, D. Walker, and R. C. Whaley. *ScaLAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1997.

[18] Boost C++ Libraries. https://www.boost.org/.

[19] C. Burstedde, L. C. Wilcox, and O. Ghattas. p4est: Scalable algorithms for parallel adaptive mesh refinement on forests of octrees. *SIAM J. Sci. Comput.*, 33(3):1103–1133, 2011.

[20] T. C. Clevenger, T. Heister, G. Kanschat, and M. Kronbichler. A flexible, parallel, adaptive geometric multigrid method for FEM. *ACM Transactions on Mathematical Software*, 47(1):7, Dec. 2021.

[21] T. A. Davis. Algorithm 832: UMFPACK V4.3—an unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software*, 30:196–199, 2004.

[22] D. Davydov, T. Gerasimov, J.-P. Pelteret, and P. Steinmann. Convergence study of the *h*-adaptive PUM and the *hp*-adaptive FEM applied to eigenvalue problems in quantum mechanics. *Advanced Modeling and Simulation in Engineering Sciences*, 4(1):7, Dec 2017.

[23] L. Demkowicz and J. Gopalakrishnan. A class of discontinuous Petrov–Galerkin methods. Part I: The transport equation. *Computer Methods in Applied Mechanics and Engineering*, 199(23-24):1558–1572, Apr. 2010.

[24] L. Demkowicz and J. Gopalakrishnan. A class of discontinuous Petrov–Galerkin methods. II. Optimal test functions. *Numerical Methods for Partial Differential Equations*, 27(1):70–105, Jan. 2011.

[25] L. Demkowicz, J. Gopalakrishnan, and A. H. Niemi. A class of discontinuous Petrov–Galerkin methods. Part III: Adaptivity. *Applied Numerical Mathematics*, 62(4):396–427, Apr. 2012.

[26] A. DeSimone, L. Heltai, and C. Manigrasso. Tools for the Solution of PDEs Defined on Curved Manifolds with deal.II. Technical Report 42/2009/M, SISSA, 2009.

[27] M. Fehling and W. Bangerth. Algorithms for parallel generic *hp*-adaptive finite element software. *ACM Transactions on Mathematical Software*, 49(3):25, 2023.

[28] S. Filippone and M. Colajanni. PSBLAS: A library for parallel linear algebra computation on sparse matrices. *ACM Transactions on Mathematical Software*, 26(4):527–550, 2000.

[29] M. Galassi, J. Davies, J. Theiler, B. Gough, G. Jungman, M. Booth, and F. Rossi. *GNU Scientific Library Reference Manual*. Network Theory Ltd., 3rd edition, 2009. https://www.gnu.org/software/gsl.

[30] D. J. Gardner, D. R. Reynolds, C. S. Woodward, and C. J. Balos. Enabling new flexibility in the sundials suite of nonlinear and differential/algebraic equation solvers. *ACM Transactions on Mathematical Software (TOMS)*, 48(3):31, 2022.

[31] R. Gassmöller, H. Lokavarapu, E. Heien, E. G. Puckett, and W. Bangerth. Flexible and scalable particle-in-cell methods with adaptive mesh refinement for geodynamic computations. *Geochemistry, Geophysics, Geosystems*, 19(9):3596–3604, 2018.

[32] C. Geuzaine and J.-F. Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre-and post-processing facilities. *International journal for numerical methods in engineering*, 79(11):1309–1331, 2009.

[33] N. Giuliani, A. Mola, and L. Heltai. π-BEM: A flexible parallel implementation for adaptive, geometry aware, and high order boundary element methods. *Advances in Engineering Software*, 121:39–58, July 2018.

[34] A. Griewank, D. Juedes, and J. Utke. Algorithm 755: ADOL-C: a package for the automatic differentiation of algorithms written in C/C++. *ACM Transactions on Mathematical Software*, 22(2):131–167, 1996.

[35] L. Heltai, W. Bangerth, M. Kronbichler, and A. Mola. Propagating geometry information to finite element computations. *ACM Transactions on Mathematical Software*, 47(4):32, 2021.

[36] L. Heltai and A. Mola. Towards the Integration of CAD and FEM using open source libraries: a Collection of deal.II Manifold Wrappers for the OpenCASCADE Library. Technical report, SISSA, 2015.

[37] T.-W. Huang, D.-L. Lin, C.-X. Lin, and Y. Lin. Taskflow: A lightweight parallel and heterogeneous task graph computing system. *IEEE Transactions on Parallel and Distributed Systems*, 33(6):1303–1320, 2021.

[38] International Standards Organization. ISO/IEC 14882:2026: The C++26 programming language standard, working draft. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/n5046.pdf, 2026.

[39] B. Janssen and G. Kanschat. Adaptive multilevel methods with local smoothing for H^1- and H^{curl}-conforming high order finite element methods. *SIAM J. Sci. Comput.*, 33(4):2095–2114, 2011.

[40] A. Javeed, D. P. Kouri, D. Ridzal, and G. Von Winckel. Get ROL-ing: An introduction to Sandia's rapid optimization library. In *7th International Conference on Continuous Optimization*, 2022.

[41] G. Kanschat. Multi-level methods for discontinuous Galerkin FEM on locally refined meshes. *Comput. & Struct.*, 82(28):2437–2445, 2004.

[42] G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM J. Sci. Comput.*, 20(1):359–392, 1998.

[43] M. Kronbichler and K. Kormann. A generic interface for parallel cell-based finite element operator application. *Comput. Fluids*, 63:135–147, 2012.

[44] M. Kronbichler and K. Kormann. Fast matrix-free evaluation of discontinuous Galerkin finite element operators. *ACM Transactions on Mathematical Software*, 45(3):29, 2019.

[45] R. B. Lehoucq, D. C. Sorensen, and C. Yang. *ARPACK users' guide: solution of large-scale eigenvalue problems with implicitly restarted Arnoldi methods*. SIAM, Philadelphia, 1998.

[46] List of changes for `deal.II` release 9.8. https://dealii.org/developer/doxygen/deal.II/changes_between_9_7_1_and_9_8_0.html.

[47] Magic Enum C++. https://github.com/Neargye/magic_enum.

[48] M. Maier, M. Bardelloni, and L. Heltai. LinearOperator – a generic, high-level expression syntax for linear algebra. *Computers and Mathematics with Applications*, 72(1):1–24, 2016.

[49] M. Maier, M. Bardelloni, and L. Heltai. LinearOperator Benchmarks, Version 1.0.0, 2016.

[50] M. Mayr, A. Heinlein, C. Glusa, S. Rajamanickam, M. Arnst, R. A. Bartlett, L. Berger-Vergiat, E. G. Bowman, K. Devine, G. Harper, M. Heroux, M. Hoemmen, J. J. Hu, B. Kelley, K. Kim, D. P. Kouri, P. Kuberry, K. Liegeois, C. C. Ober, R. P. Pawlowski, C. Pearson, M. Perego, E. Phipps, D. Ridzal, N. V. Roberts, C. M. Siefert, H. K. Thornquist, R. Tomasetti, C. R. Trott, R. S. Tuminaro, J. M. Willenbring, M. M. Wolf, and I. Yamazaki. Trilinos: Enabling scientific computing across diverse hardware architectures at scale. *ACM Transactions on Mathematical Software*, 52(2):12, June 2026.

[51] P. Munch, T. Heister, L. Prieto Saavedra, and M. Kronbichler. Efficient distributed matrix-free multigrid methods on locally refined meshes for FEM computations. *ACM Transactions on Parallel Computing*, 10(1):3, 2023.

[52] muparser: Fast Math Parser Library. https://beltoforion.de/en/muparser.

[53] OpenCASCADE: Open CASCADE Technology, 3D modeling & numerical simulation. https://www.opencascade.org/.

[54] A. Prokopenko, D. Lebrun-Grandié, D. Arndt, and B. Turcksin. Arborx 2.0, 04 2025.

[55] J. Reinders. *Intel Threading Building Blocks*. O'Reilly Media, 2007.

[56] J. E. Roman, F. Alvarruiz, C. Campos, L. Dalcin, P. Jolivet, and A. Lamas Daviña. Improvements to SLEPc in releases 3.14–3.18. *ACM Transactions on Mathematical Software*, 49(3):29, 2023.

[57] A. Sartori, N. Giuliani, M. Bardelloni, and L. Heltai. deal2lkit: A toolkit library for high performance programming in deal.II. *SoftwareX*, 7:318–327, 2018.

[58] W. Schroeder, K. Martin, and B. Lorensen. *The Visualization Toolkit (4th ed.)*. Kitware, 2006. https://vtk.org/.

[59] T. Schulze, A. Gessler, K. Kulling, D. Nadlinger, J. Klein, M. Sibly, and M. Gubisch. Open asset import library (assimp). https://github.com/assimp/assimp, 2021.

[60] SymEngine: fast symbolic manipulation library, written in C++. https://symengine.org/.

[61] The CGAL Project. *CGAL User and Reference Manual*. CGAL Editorial Board, 6.0.1 edition, 2024.

[62] The HDF Group. Hierarchical Data Format, version 5, 2025. https://www.hdfgroup.org/solutions/hdf5/.

[63] The Trilinos Project Team. *The Trilinos Project*. Linux Foundation / High Performance Software Foundation. https://trilinos.github.io/.

[64] C. R. Trott, D. Lebrun-Grandié, D. Arndt, J. Ciesko, V. Dang, N. Ellingwood, R. Gayatri, E. Harvey, D. S. Hollman, D. Ibanez, N. Liber, J. Madsen, J. Miles, D. Poliakoff, A. Powell, S. Rajamanickam, M. Simberg, D. Sunderland, B. Turcksin, and J. Wilke. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, 2022.

[65] B. Turcksin, M. Kronbichler, and W. Bangerth. *WorkStream* – a design pattern for multicore-enabled finite element computations. *ACM Transactions on Mathematical Software*, 43(1):2, 2016.

[66] M. Wolff et al. heaptrack - a heap memory profiler for Linux. https://github.com/KDE/heaptrack. Accessed: 2026-07-06.

[67] J. Zarestky, M. Bigler, M. Brazile, T. Lopes, and W. Bangerth. Reflective writing supports metacognition and self-regulation in graduate computational science and engineering. *Computers and Education Open*, 3:100085/1–12, 2022.

[68] zlib. https://zlib.net.